Bachelorarbeit

ENTWURF UND UMSETZUNG EINER GRAPHBASIERTEN MULTI-AGENTENARCHITEKTUR ZUR VERBESSERTEN GENERIERUNG VON BLOGBEITRÄGEN MIT LARGE LANGUAGE MODELS UNTER NUTZUNG VON RETRIEVAL-AUGMENTED GENERATION

Verfasser

Marc Rodenbäck

angestrebter akademischer Grad

Bachelor of Science (BSc)

Wien, 2026

Studienkennzahl lt. Studienblatt:	UA 033 526
Fachrichtung:	Informatik - Wirtschaftsinformatik
Betreuer:	Dipl.-Ing. Dr.techn. Marian Lux

Inhaltsverzeichnis

1 Einleitung **4**

2 Gap-Analyse **4**
2.1 Backend 4
2.2 Frontend 5

3 Theoretische Grundlagen **5**
3.1 Large Language Models und Prompt-Engineering 5
3.2 Retrieval-Augmented Generation 6
3.3 Multi-Agenten-Systeme und Graph-Architekturen 7
3.4 Prinzipien des UX/UI-Designs 7

4 Systemarchitektur und Implementierung **8**
4.1 Konzeption der Gesamtarchitektur 8
4.2 Backend: LangGraph-Workflow und Agenten-Rollen 10
4.2.1 Zustandsverwaltung 13
4.2.2 Die Fact-Checker-Loop 13
4.3 Frontend: Telegram-Interface 14

5 Methodisches Vorgehen **16**
5.1 Frontend-First-Ansatz 16
5.2 Framework-Wechsel: Von CrewAI zu LangGraph 17
5.3 KI-gestützte Entwicklung 18

6 Evaluierung **19**
6.1 Definition und Durchführung der Testreihe 19
6.2 Backend 21
6.3 Frontend 22

7 Adaptation Manual **22**
7.1 Schnelle Orientierung 23
7.2 Anpassung der Konfiguration (config-Ordner) 23
7.2.1 `config/configs.yaml` - Globale Laufzeiteinstellungen . . 23
7.2.2 `config/agents.yaml` - Agentenrollen, Persona, Einschränkungen 23
7.2.3 `config/tasks.yaml` - Schritt-Outputs & Lieferobjekte . . 24
7.3 Anpassung des Workflows (LangGraph) 24
7.3.1 Ändern des Agenten-Ablaufs 24
7.3.2 Ändern der Revisionslogik 25
7.4 Anpassung des Agentenverhaltens 25
7.4.1 Der gemeinsame Zustand (Informationsfluss) 25
7.4.2 Verbesserungen des Writers (wie Neuschreibungen implementiert werden) 26
7.5 Modellanpassung (`llm/factory.py`) 26
7.5.1 Wechseln der LLM-Modelle (Ollama) 26

7.5.2 Verwendung von Nicht-Ollama-Modellen 26
7.6 RAG-Anpassung 27
7.6.1 Anpassung des Vektorspeichers (Chroma) 27
7.6.2 Anpassung der Websuche 27
7.7 Anpassung des Telegram-Chatbots 28

8 Fazit und Ausblick **28**
8.1 Zusammenfassung der Ergebnisse 28
8.2 Ausblick 29

Zusammenfassung

Diese Bachelorarbeit befasst sich mit der Neuentwicklung und Evaluierung eines agentenbasierten Chatbots zur automatisierten Erstellung von Blogposts. Eine erste Gap-Analyse der ursprünglichen Umsetzung des Prototyps mit CrewAI deckte fundamentale Schwächen auf, wie mangelnde Transparenz, schwerfällige Bedienbarkeit und Schwierigkeiten bei der Wartung und Erweiterung. Zur Lösung dieser Limitierungen wurde das System auf eine graphenbasierte Architektur mit LangGraph migriert, woraus eine modulare 4-Schichten-Architektur hervorging. Im Backend ermöglicht ein deterministischer Graph nun die Selbstreflexion der Agenten, während das Frontend als interaktiver Telegram-Bot mit Status-Streaming in Echtzeit realisiert wurde.

1 Einleitung

Die rasante Entwicklung der Künstlichen Intelligenz und ihrer mannigfaltigen Einsatzmöglichkeiten hat nicht nur in der akademischen Lehre, sondern auch in der unternehmerischen Praxis einen fundamentalen Umbruch bewirkt. Moderne Sprachmodelle werden zunehmend als High-Tech-Werkzeuge in Geschäftsprozesse integriert [4]. Das Hauptziel dieser Arbeit besteht daher in der Konzeption, Entwicklung und Evaluierung einer fähigen Multi-Agenten-Architektur. Die Basis ist ein bestehender, auf dem Framework CrewAI basierender Chatbot-Prototyp, der Blogbeiträge generieren soll. Wie die praktische Anwendung aber zeigte, stieß dieser Prototyp an Grenzen hinsichtlich Flexibilität, Effizienz, Wartbarkeit und User Experience (UX). Um diese Einschränkungen abzubauen, wurde das System im Rahmen dieser Arbeit auf das graphenbasierte Framework LangGraph migriert. Ziel ist es, einen expliziten, deterministischen Workflow zu erschaffen, der falsche Annahmen der Modelle unterdrückt und sich strikt an Einschränkungen durch den Nutzer hält. Ergänzend zur Restrukturierung des Backends, wird außerdem ein benutzerorientiertes Interface in Telegram umgesetzt, welches die bisherigen textbasierten Eingaben durch eine einladende Oberfläche mit Status-Updates und Button-Navigation ersetzt.

2 Gap-Analyse

Um der Weiterentwicklung des ursprünglichen Tools eine Richtung zu geben, wurde es im Vorfeld einer detaillierten Gap-Analyse unterzogen. Dabei wurden starke Defizite ermittelt, welche die Anwendung und die Skalierbarkeit des Systems stark einschränkten. Diese Lücken lassen sich in 2 Bereiche unterteilen: Architektonische Schwächen im Backend und eine kontraintuitive Bedienung im Frontend.

2.1 Backend

Das Framework CrewAI zeichnet sich durch einen hohen Abstraktionsgrad aus [2]. Dies führte in der Praxis dazu, dass der interne Zustand in der Vorgän-

gerversion äußerst intransparent war. Das Debugging gestaltete sich dadurch schwierig, weshalb auch der genaue Grund für die langen Durchlaufzeiten und der unter vielen Fallbeispielen wiederkehrenden Endlosschleifen nicht geklärt werden konnten. Der Bot musste somit vom Anwender häufig zum Abbruch gezwungen werden. Die Vorgängerversion nutzte außerdem ein einziges Sprachmodell für alle Agenten (Llama 3.1). Dies führte zu Ineffizienzen, da Modelle nicht entsprechend ihrer spezifischen Stärken eingesetzt wurden.

Die größte Hürde zeigte sich allerdings in Form der mangelhaften Wartbarkeit. Beim Versuch CrewAI auf eine modernere Version zu heben, zeigte sich ein vermeintlicher Zwang zur Nutzung einer Schnittstelle zu OpenAI. Auf diese neue Einschränkung haben auch andere Entwickler in Foreneinträgen hingewiesen. [3] Die strikte Anforderung, das System vollständig lokal und open-source zu betreiben, ließ sich dadurch nicht länger einhalten.

2.2 Frontend

Die Konfiguration des Blogposts (wie Länge, Tonalität oder Zielgruppe) musste vom Anwender mühsam und fehleranfällig als Freitext in den Chat geschrieben werden. Es fehlte eine aktive Nutzerführung, Textnachrichten vom Bot waren wegen schlechter Struktur und fehlender Highlightings nur schwer lesbar. Diese Umstände haben die Bedienbarkeit erheblich erschwert. Die Generierung eines fertigen Blogposts nahm aufgrund der erwähnten Unzulänglichkeiten des Backends teils bis zu über einer Stunde in Anspruch. Der alte Prototyp ließ den Nutzer in dieser Zeit mit einer statischen Rückmeldung ("processing...“) im Ungewissen über den Fortschritt und den aktuellen Systemstatus.

3 Theoretische Grundlagen

3.1 Large Language Models und Prompt-Engineering

Large Language Models (LLMs) zeigen den aktuellen Stand der Technik im Bereich der Verarbeitung natürlicher Sprache (Natural Language Processing) auf. Die grundlegende Funktionsweise dieser auf künstlichen neuronalen Netzen basierenden Modelle beruht auf der statistischen Vorhersage von Textbausteinen, sogenannten Tokens. Anstatt Sprache auf einer rein regelbasierten oder semantischen Ebene zu verstehen, berechnen LLMs auf Basis der vorangegangenen Sequenz die mathematische Wahrscheinlichkeit für das jeweils nächste Token und generieren so iterativ zusammenhängende Texte [16].

Eine technische Einschränkung bei der Arbeit mit LLMs bildet das sogenannte Kontextfenster (Context Window). Dieses definiert die maximale Anzahl an Tokens, die das Modell in einem einzigen Durchlauf verarbeiten kann. Da das Modell kein persistentes Gedächtnis über diese Grenze hinaus besitzt, müssen alle für eine Aufgabe relevanten Informationen, Instruktionen und bisherigen Konversationsverläufe zwingend innerhalb dieses Limits übergeben werden. Dies macht eine effiziente Steuerung und Begrenzung der Eingabedaten -

insbesondere bei komplexen Workflows - zwingend erforderlich [16].

Um das Verhalten von LLMs effektiv zu steuern, kommt das sogenannte Prompt-Engineering zum Einsatz. Darunter versteht man die systematische Konzeption und semantische Programmierung von Eingabeaufforderungen, um die Ausgabequalität der Modelle zu maximieren. Während einfache Instruktionen bei komplexen Logik- oder Textgenerierungsaufgaben häufig zu inkorrekten oder fehlerhaften Aussagen (Halluzinationen) führen, können fortgeschrittene Techniken die Zuverlässigkeit des Systems merklich erhöhen. Ein wesentlicher Durchbruch in diesem Bereich ist das Chain-of-Thought (CoT) Prompting. Bei dieser Methode wird das Modell durch spezifische Anweisungen dazu gezwungen, seine Problemlösung nicht direkt als finales Ergebnis auszugeben, sondern den Weg dorthin in transparente, nachvollziehbare Zwischenschritte zu zerlegen. Durch dieses erzwungene, schrittweise "Nachdenken"vor der eigentlichen Textgenerierung wird die Fehleranfälligkeit des Modells reduziert, da sich Schlussfolgerungen auf den explizit generierten Kontext der vorherigen Schritte beziehen. [14] In dieser Arbeit bildet die genannte Methodik das Fundament für die Strukturierung der Workflows, um qualitativ hochwertige Blogposts zu generieren.

3.2 Retrieval-Augmented Generation

Moderne Large Language Models verfügen über eine gewaltige Wissensbasis, welche während ihrer Trainingsphase in den Modellparametern (parametrisches Gedächtnis) gespeichert wurde. Dennoch weisen sie zwei fundamentale Limitierungen auf: Ihr Wissensstand ist nach Abschluss des Trainings statisch und sie tendieren bei Wissenslücken zu plausibel klingenden, aber inkorrekten Aussagen (Halluzinationen). Um diese Schwächen zu überwinden, wurde das Konzept der Retrieval-Augmented Generation (RAG) entwickelt. RAG erweitert die KI-gestützte Textgenerierung um einen externen Abruf von Informationen aus anderen Wissensquellen (nicht-parametrisches Gedächtnis) [9].

Der typische Workflow bei RAG wird nach drei Phasen unterschieden: Indexierung, Abruf und Generierung. Zunächst werden im Rahmen der Indexierung externe Quellen, wie lokale Dokumente oder Web-Inhalte, in kleinere Textabschnitte unterteilt. Diese "Chunks"werden anschließend durch ein Embedding-Modell in Vektoren transformiert und in einer Vektordatenbank (wie beispielsweise ChromaDB) gespeichert. Das Retrieval-System sucht daraufhin in der Datenbank nach den Vektoren mit der höchsten mathematischen Übereinstimmung (meist über die Kosinus-Ähnlichkeit). Somit werden die relevantesten Textabschnitte identifiziert [6].

In der letzten Phase der Generierung werden die Informationen als zusätzlicher Kontext in den Prompt des LLMs geladen. Das Modell beantwortet dann nicht mehr nur aus seinem Trainingswissen, sondern wird auf die bereitgestellten Fakten geerdet (Grounded Generation) [9]. Für die Architektur der vorliegenden Arbeit ist RAG eine essenzielle Technologie: Durch die Kombination aus Dokumenten-Retrieval und Websuchen wird sichergestellt, dass die autonomen Agenten (insbesondere in der Rolle des Researchers) fundiert recherchieren können und die finalen Blogposts auf einer validen Faktenbasis beruhen.

3.3 Multi-Agenten-Systeme und Graph-Architekturen

Die Lösung komplexerer Aufgaben ist für isolierte Large Language Models oftmals problematisch. Die Vermischung von Informationsbeschaffung, der Strukturierung und kreativer Textgenerierung in einem einzigen Prompt überfordert das System gerne, es wird also fehleranfällig. Einen Lösungsansatz für diese Problematik bilden Multi-Agenten-Systeme (MAS). Dabei wird eine umfassende Aufgabe in modulare Teilbereiche zerlegt und an spezialisierte KI-Agenten mit klar definierten Rollen, wie etwa *Researcher*, *Editor* oder *Writer*, delegiert [15]. Durch diese Arbeitsteilung können die Modelle gezielter auf ihre jeweilige Teilaufgabe fokussiert und besser mit passenden Modellen und Prompts ausgestattet werden.

Moderne MAS sind in der Lage Feedbackschleifen zu implementieren. Anstatt Text streng linear und ohne qualitative Kontrollinstanz zu generieren, evaluieren sich die Agenten selbst. Möglich wird das z.B. durch einen zyklischen Prozess, bei dem ein generierender Agent (Actor) einen Entwurf erstellt, welcher von einem bewertenden Agenten (Evaluator) kritisiert wird, woraufhin der Actor seinen Output selbstständig korrigiert [13]. Dieser Mechanismus ist dazu da, um Halluzinationen zu verringern und bildet die theoretische Basis für die in dieser Arbeit entwickelte *Fact-Checker-Loop*.

Um solch komplexe Workflows technisch robust und transparent abzubilden, etablieren sich zunehmend zustandsbasierte Graph-Architekturen. Solche Systeme (wie beispielsweise LangGraph) definieren den KI-Workflow explizit als gerichteten Graphen. Hierbei repräsentieren die Knoten (Nodes) die ausführenden Agenten, während die Kanten (Edges) den Kontrollfluss und die Bedingungen definieren. Ein zentrales Zustandsobjekt (State) wird dabei kontinuierlich zwischen den Knoten weitergegeben und aktualisiert. Dadurch wird eine feingranulare Kontrolle über den Datenfluss ermöglicht, sowie eine transparente Fehlerverfolgung und die nahtlose Einbindung von Korrekturschleifen [8].

3.4 Prinzipien des UX/UI-Designs

Wie Følstad und Brandtzæg bereits 2017 aufzeigten, vollzieht sich in der Human-Computer Interaction (HCI) ein Paradigmenwechsel: Klassische grafische Benutzeroberflächen werden zunehmend durch dialogorientierte Systeme, sogenannte Conversational User Interfaces (CUIs), ergänzt oder abgelöst [5]. Während diese textbasierten Schnittstellen eine natürliche und intuitive Kommunikation versprechen, bergen sie spezifische Herausforderungen für die Usability: Ein zentrales Problem vieler textbasierter KI-Systeme ist die mangelnde Handlungsaufforderung offener Texteingabefelder. Nutzer stehen häufig vor der Schwierigkeit, die Fähigkeiten und Grenzen des Systems nicht richtig zu verstehen. Eine Studie von Luger und Sellen (2016) belegt, dass die Diskrepanz zwischen Nutzererwartung und tatsächlicher Systemleistung, kombiniert mit der Unklarheit darüber, welche Befehle das System verarbeiten kann, zu kognitiver Überlastung und Frustration führt [10]. Um diese Hürden zu überwinden, etablieren sich spezifische Design-Richtlinien für die Interaktion mit KI-Systemen. Eine

fundamentale Regel, wie sie beispielsweise in den *Guidelines for Human-AI Interaction* postuliert wird, lautet: "Make clear what the system can do." [1].

Die vorliegende Arbeit versucht, diese Probleme gezielt im Frontend-Design via Telegram anzugehen. Für die Konzeption bedeutet dies, dass der Nutzer nicht durch ein leeres Textfeld alleingelassen werden darf. Stattdessen sollten Interaktionsmöglichkeiten durch strukturierte UI-Elemente wie z.B. vordefinierte Auswahlmenüs aktiv geführt werden. Durch den Ersatz mühsamer Texteingaben durch flüssige Button-Navigation und die Implementierung eines Live-Status-Streamings wird die Ungewissheit eliminiert und der interne Workflow der Agenten für den Nutzer jederzeit nachvollziehbar visualisiert.

4 Systemarchitektur und Implementierung

In diesem Kapitel wird die technische Konzeption und praktische Umsetzung der Architektur vorgestellt. Der vollständige Code des Projekts ist in einem öffentlichen GitHub-Repository dokumentiert [12].

4.1 Konzeption der Gesamtarchitektur

Um den in der Gap-Analyse identifizierten Schwächen der Vorgängerversion entgegenzuwirken, wurde das System von Grund auf neu designet. Die zentrale architektonische Leitlinie bildet dabei die strikte Trennung von Zuständigkeiten. Die Applikation ist in eine modulare 4-Schichten-Architektur unterteilt, um Konfiguration, Workflow-Logik, Modellauswahl und das Benutzerinterface (UI) bestmöglich voneinander zu entkoppeln (vgl. Abbildung 1).

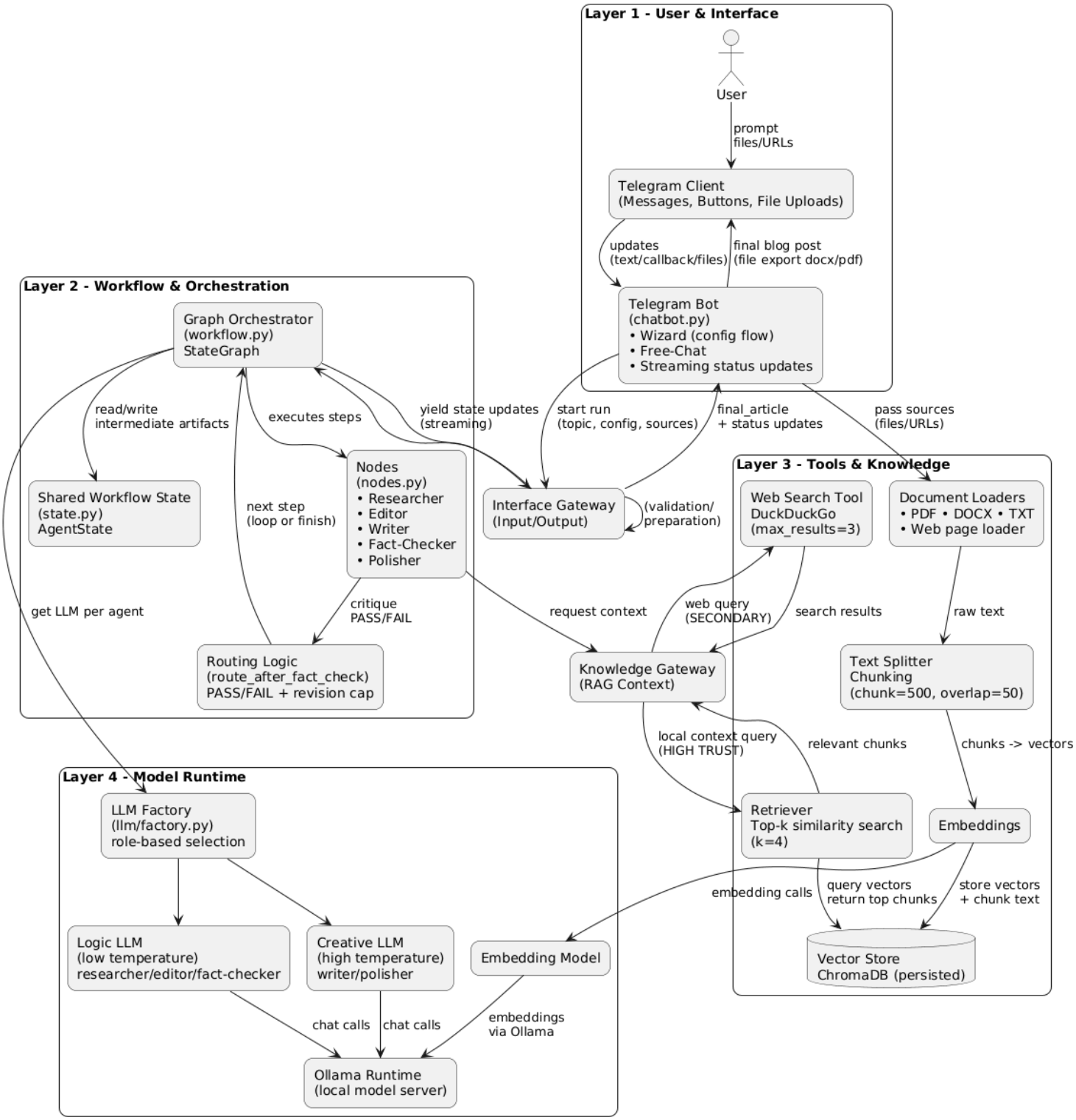

Abbildung 1: Schichtenarchitektur des Systems

Die Architektur wurde in verschiedene Ebenen gegliedert:

- **Layer 1 - User & Interface:** Diese Schicht kapselt die Interaktion mit dem Endanwender. Sie nutzt einen Telegram-Bot als Frontend, um Eingaben (Prompts, Konfigurationen, Dateiuploads) zu speichern und sowohl den Verarbeitungsstatus via Live-Streaming, als auch das fertige Ergebnis als Markdown-Datei zurückzugeben.

- **Layer 2 - Workflow & Orchestration:** Das Herzstück des Systems bildet der in LangGraph implementierte Graph. Dieser orchestriert die spezialisierten Agenten-Knoten (*Researcher, Editor, Writer, Fact-Checker, Polisher*) und verwaltet den Systemzustand in *Agent State*. Zudem ist hier die Routing-Logik verankert, die den Ablauf hinter dem Fact-Checker steuert.

- **Layer 3 - Tools & Knowledge:** Diese Schicht stellt die externe Informationsbeschaffung im Sinne der RAG sicher. Sie umfasst das Laden und Verarbeiten lokaler Dokumente, deren Vektorisierung und Speicherung in ChromaDB, sowie eine begleitende Websuche zur Verifizierung von Fakten.

- **Layer 4 - Model Runtime:** Die unterste Ebene abstrahiert die Anbindung der Large Language Models. Durch eine dedizierte *LLM Factory* werden je nach Agentenrolle spezifische Modelle (Logik vs. Kreativ) zugewiesen und über eine lokale Ollama-Laufzeitumgebung ausgeführt.

Durch die Implementierung wird die explizite Trennung der Schichten sichergestellt, sodass beispielsweise ein Austausch des Frontends (Layer 1) oder ein Wechsel der LLM-Provider (Layer 4) durchgeführt werden kann, ohne die Workflow-Logik (Layer 2) modifizieren zu müssen.

4.2 Backend: LangGraph-Workflow und Agenten-Rollen

Die logische Orchestrierung der Agenten erfolgt im Backend über das Framework LangGraph. Wie in Abbildung 2 detailliert dargestellt, wird der Prozess der Blogpost-Generierung als gerichteter Graph modelliert. Die Knoten (Nodes) repräsentieren dabei die LLM-Agenten, während die Kanten (Edges) den Datenfluss und die logischen Abhängigkeiten definieren. Die Architektur vereint die Vorverarbeitung von Wissensquellen nahtlos mit der eigentlichen Agenten-Orchestrierung. Der Prozessaufbau lässt sich anhand des Graphen in folgende Phasen unterteilen:

- **Initialisierung und Pre-Processing:** Zunächst übergibt der Nutzer via Telegram die Konfiguration (*Topic, Audience, Tone, Length*) sowie externe Quellen. Lokale Dokumente durchlaufen eine RAG-Pipeline (*Source Loading, Chunking, Embeddings*) und werden im ChromaDB Vector Store abgelegt.

- **Die Agenten-Pipeline:** Der LangGraph-Workflow startet mit dem *Researcher Agent*. Dieser zieht relevante Chunks aus der Vektordatenbank und führt autonome Websuchen (via DuckDuckGo) durch, um strukturierte Recherchedaten zu generieren. Auf dieser fundierten Basis erstellt der *Editor Agent* eine inhaltliche Gliederung, welche der *Writer Agent* anschließend in einen ersten Textentwurf ausformuliert.

- **Qualitätskontrolle und Routing:** Der Entwurf wird an den *Fact-Checker Agent* übergeben, der eine Kritik samt „PASS“- oder „FAIL“-Bewertung erzeugt. Ein deterministischer Entscheidungsknoten wertet dieses Ergebnis aus: Bei einem „FAIL“ wird der Text über eine Revisionsschleife zur Überarbeitung an den Writer Agent zurückgewiesen. Um Endlosschleifen zu verhindern, ist dieser Zyklus auf maximal zwei Iterationen limitiert.

- **Finalisierung:** Bei einem „PASS“ (oder dem Erreichen des Revisionslimits) übernimmt der *Polisher Agent*. Er finalisiert den Artikel und liefert diesen mitsamt dem kontinuierlichen Status-Streaming an das Telegram-Interface des Nutzers zurück.

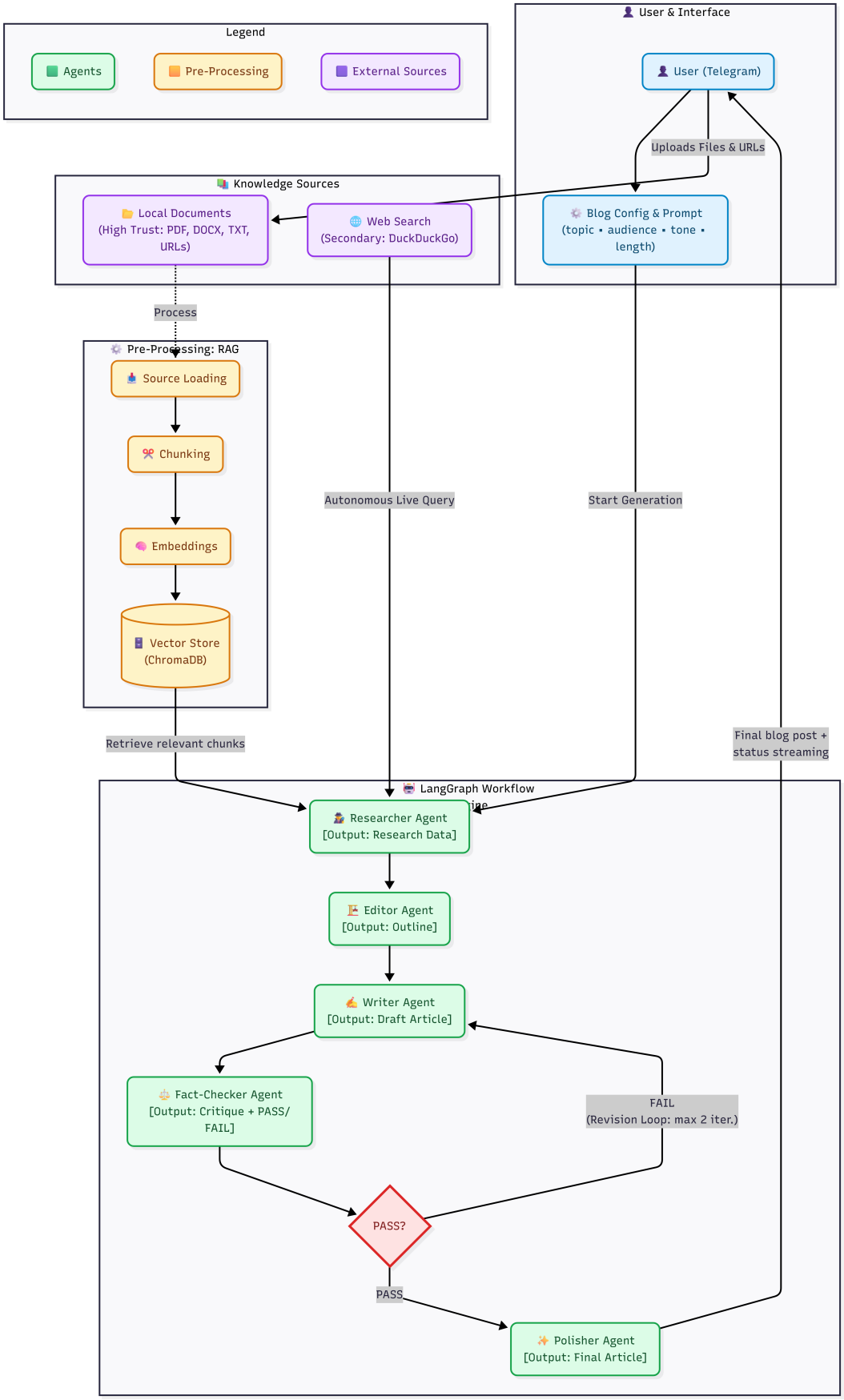

Abbildung 2: Detaillierter LangGraph-Workflow der Agenten

4.2.1 Zustandsverwaltung

Ein zentraler Unterschied zu intransparenten Frameworks wie CrewAI ist die Möglichkeit der expliziten Kontrolle der Zustände in LangGraph [7]. Alle Agenten arbeiten auf einem geteilten Zustandsobjekt (*AgentState*), das bei jedem Schritt aktualisiert und an den nächsten Knoten weitergereicht [12]. Dies garantiert maximale Transparenz und ermöglicht ein präzises Debugging. Der Zustand definiert exakt, welche Daten während des Durchlaufs erzeugt werden müssen:

```python
class AgentState(TypedDict):
    """
    State of blogpost-workflow. Saves data, that is shared between the agents.
    """

    topic: str
    target_len: str
    language_level: str
    information_level: str
    language: str
    tone: str
    additional_info: str
    source_documents: List[str]
    research_data: List[str]
    outline: List[str]
    draft: str
    critique: Optional[str]
    final_article: str
    revision_count: int
    current_status: str
```

Abbildung 3: Definition von AgentState

Der *Researcher-Agent* befüllt diesen State zu Beginn mit gefundenen Fakten (*research_ data*) , indem er die RAG-Pipeline abfragt. Daraufhin gestaltet der *Editor-Agent* durch den Inhalt eine klare Struktur (*outline*), bevor der *Writer-Agent* den ersten Textentwurf (*draft*) generiert.

4.2.2 Die Fact-Checker-Loop

Um das Risiko von Halluzinationen zu minimieren und die Textqualität zu steigern, wurde ein *Fact-Checker-Agent* in den Workflow eingebaut, der den vom *Writer* erstellten Entwurf kritisch gegen die gefundenen Recherche-Daten evaluiert und ein ”PASS” oder ”FAIL” (im Fall "FAIL"mit konstruktiver Kritik) (*critique*) zurückgibt. Hier unterscheidet sich die Architektur vom linearen Verarbeitungsfluss der originalen Implementierung mit CrewAI: Durch eine Bedingung wird eine Feedback-Schleife implementiert [12]. Die zugehörige Routing-Funktion entscheidet dynamisch, ob der Graph zum Polisher voranschreitet oder den Text zur Überarbeitung an den *Writer* zurückschiebt:

```python
def route_after_fact_check(state: AgentState) -> str:
    """
    Decides if text goes back to writer or forwards it to polisher.
    """

    if state.get("revision_count", 0) >= 2:
        return "polisher"

    critique = state.get("critique", "").strip().upper()

    if critique == "PASS" or critique.startswith("PASS") or critique == "":
        return "polisher"
    else:
        return "writer"
```

Abbildung 4: Routing-Logik nach der Faktenprüfung

Durch die Deckelung auf maximal zwei Revisionsschleifen (`revision_count >= 2`) wird verhindert, dass sich der Prozess in Endlosschleifen verfängt. Erst wenn die Faktenprüfung "PASS"gibt oder das Limit erreicht wird, finalisiert der *Polisher-Agent* den Text, wendet das gewünschte Wording an und bereitet die Markdown-Formatierung für die Ausgabe vor.

4.3 Frontend: Telegram-Interface

Während der logische Aufbau des Systems im LangGraph-Backend liegt, erfordert die Interaktion mit dem Endnutzer ein intuitives und transparentes Frontend. Wie in der theoretischen Konzeption (vgl. Kapitel 3.4) dargelegt, scheitern viele textbasierte KI-Systeme an mangelnder Systemtransparenz und fehlender Führung des Users. Um diese Hürden zu überwinden, wurde das Frontend der Anwendung als interaktiver Telegram-Bot realisiert. Anstatt den Nutzer mit einem leeren Textfeld zu konfrontieren, führt das System durch einen konfigurierbaren "Wizard". Über vordefinierte Buttons können Parameter wie Tonalität, Zielgruppe und Länge des Posts intuitiv ausgewählt werden, was die Bedienbarkeit deutlich verbessert [12].

Eine wesentliche Herausforderung bei der Orchestrierung mehrerer Agenten ist die Verarbeitungsdauer, besonders da ein vollständiger Recherche- und Schreibzyklus mehrere Minuten in Anspruch nehmen kann. Um den von Luger und Sellen (2016) beschriebenen Vertrauensverlust durch Intransparenz zu verhindern, zeigt das Frontend ein kontinuierliches Live-Streaming der Zustände, in denen sich der Prozess zum jeweiligen Zeitpunkt befindet.

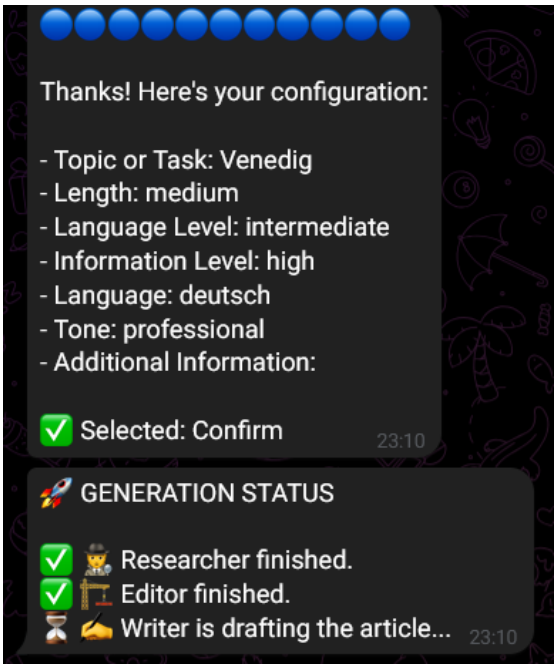

Abbildung 5: Telegram-Chat mit Status-Visualisierung

Sobald der Workflow im Graph gestartet wird, generiert das Backend bei jedem Übergang zwischen den Knoten ein Signal, dieses wird durch die Schnittstelle zum Interface empfangen, welches die bestehende Telegram-Nachricht entsprechend aktualisiert (vgl. Abbildung 5). Dies wird konkret über die Methode `edit_message_text` der Telegram-API gelöst. Der Nutzer sieht so in Echtzeit, welcher Agent (z.B. *Researcher* oder *Fact-Checker*) aktuell arbeitet. Ein Code-Ausschnitt zeigt diesen asynchronen Update-Mechanismus (vgl. Abbildung 6). Besonders zentral ist die Auswertung nach dem *Fact-Checker* (ab Zeile 1140): Das System liest die inhaltliche Bewertung (`critique`) und die bisherige Revisionsanzahl (`rev_count`) direkt aus dem Graph-Status aus. Bei einer Freigabe (`"PASS"`) wechselt die Statusanzeige zum *Polisher*. Bei Fehlern greift hingegen der `else`-Zweig und informiert den Nutzer in Echtzeit über die Neuformulierung samt aktueller Schleifen-Nummer (`Revision {rev_count}`).:

```
1122 if node_name == "researcher":
1123     status_text += "✅ 🔬 Researcher finished.\n"
1124     await status_msg.edit_text(
1125         status_text + "⏳ 📝 Editor is creating the outline..."
1126     )
1127
1128 elif node_name == "editor":
1129     status_text += "✅ 📝 Editor finished.\n"
1130     await status_msg.edit_text(
1131         status_text + "⏳ ✍️ Writer is drafting the article..."
1132     )
1133
1134 elif node_name == "writer":
1135     status_text += "✅ ✍️ Writer finished.\n"
1136     await status_msg.edit_text(
1137         status_text + "⏳ ⚖️ Fact Checker is verifying facts..."
1138     )
1139
1140 elif node_name == "fact_checker":
1141     status_text += "✅ ⚖️ Fact Checker finished.\n"
1142     critique = state_update.get("critique", "").strip().upper()
1143     rev_count = state_update.get("revision_count", 0)
1144
1145     if (
1146         critique == "PASS"
1147         or critique.startswith("PASS")
1148         or critique == ""
1149         or rev_count >= 2
1150     ):
1151         await status_msg.edit_text(
1152             status_text
1153             + "⏳ ✨ Polisher is formatting the final text..."
1154         )
1155     else:
1156         await status_msg.edit_text(
1157             status_text
1158             + f"⚠️ ✍️ Fact Checker found errors! Writer is rewriting (Revision {rev_count})..."
1159         )
1160
1161 elif node_name == "polisher":
1162     status_text += "✅ ✨ Polisher finished.\n"
1163     await status_msg.edit_text(
1164         status_text + "🎉 Generation complete!"
1165     )
```

Abbildung 6: Update-Mechanismus im Telegram-Interface

Mit diesem Feedback wird die ursprüngliche Intransparenz des Systems beendet, wodurch der Nutzer ein unmittelbares Verständnis für die Arbeitsteilung der Agenten erhält, was die User Experience und die gefühlte Wartezeit drastisch verbessert. Nach erfolgreichem Durchlauf aller Agenten liefert der Bot den finalen Text als formatierte Markdown-Nachricht zum Download.

5 Methodisches Vorgehen

5.1 Frontend-First-Ansatz

Bevor die komplexe LangGraph-Logik entwickelt wurde, lag der erste Entwicklungsschritt in der Konzeption und Implementierung des Telegram-Bots. Dieser Ansatz bot mehrere Vorteile:

- **Definition der Systemanforderungen:** Durch das frühzeitige Design der Buttons und des Wizard-Flows in Telegram konnten Testläufe während der Backend-Entwicklung deutlich beschleunigt werden.

- **Fokus auf Transparenz:** Die Notwendigkeit eines Live-Streamings wurde direkt zu Beginn der Entwicklung als Kernanforderung identifiziert und in den finalen Schritten des Projektes schließlich durch die darauf ausgerichtete Backend-Architektur mit minimalem Aufwand umgesetzt.

- **Entkopplung:** Durch die initiale Bereitstellung einer funktionierenden UI konnte das Backend im weiteren Verlauf modular und isoliert entwickelt werden, ohne die Schnittstelle zum Nutzer zu berühren.

Erst nachdem die Interaktion zwischen Nutzer und System durch das Frontend klar definiert war, wurde die Implementierung der Backend-Logik umgesetzt.

5.2 Framework-Wechsel: Von CrewAI zu LangGraph

In der Anfangsphase des Projekts wurde zunächst überlegt, das bereits im Vorprojekt bestehende Framework CrewAI für die Orchestrierung der Agenten beizubehalten. CrewAI zeichnet sich durch eine hohe Abstraktionsebene aus, die einen schnellen Einstieg in die Multi-Agenten-Entwicklung durch ein einfaches, rollenbasiertes Setup ermöglicht. Aus Entwicklerperspektive erwies sich CrewAI als zu starr und intransparente "Blackbox". Der interne Zustand und der genaue Informationsfluss zwischen den Agenten werden weitgehend verborgen [2]. Dies erschwerte nicht nur das Debugging bei Fehlern massiv, bei Aktualisierungsversuchen zeigte sich außerdem ein scheinbar neu entstandener Zwang auf die OpenAI-API zugreifen zu müssen, was im Entwicklungsprozess zu einer unüberwindbaren Hürde für die Ansteuerung lokaler LLMs wurde. Ein Issue-Eintrag eines Entwicklers adressiert dieses Problem ebenfalls. [3]. An dieser Stelle wurde die Entscheidung getroffen, einen Wechsel auf LangGraph zu vollziehen. Trotz eines höheren Entwicklungsaufwands, zeigten sich die Vorteile von LangGraph vor allem darin, dass die Implementierung spezifischer Kontrollstrukturen, wie der zyklischen *Fact-Checker-Loop*, ermöglicht wurde.

LangGraph adressiert die erwähnten Hürden nämlich durch einen expliziten graphenbasierten Ansatz. Anstatt den Kontrollfluss dem Framework zu überlassen, muss der Entwickler den Workflow als gerichteten Graphen mit Knoten und Kanten definieren [8]. Dies bot entscheidende Vorteile für die Implementierung:

- **Explizites State-Management:** Jeder Zwischenschritt der Agenten ist in einem typisierten State-Objekt greifbar und veränderbar [8].

- **Bedingtes Routing (Conditional Edges):** Komplexe Revisionsschleifen lassen sich programmatisch exakt steuern (z. B. maximal zwei Iterationen bei einem "FAIL"des Fact-Checkers).

- **Natives Streaming:** Der Graphen-Status lässt sich problemlos abgreifen, was die zwingende Voraussetzung für die Telegram-Live-Status-Updates war.

Neben den pragmatischen Entwicklervorteilen war dieser Wechsel insbesondere aus wissenschaftlicher Perspektive in dieser Bachelorarbeit sinnvoll. Es eröffnete sich dadurch nämlich mehr Transparenz, Nachvollziehbarkeit und die Möglichkeit, Systemverhalten kausal zu erklären. Während ein abstrahiertes Framework, wie CrewAI, diese Analyse erschwert, zwingt LangGraph zur bewussten Modellierung eines deterministischen Multi-Agenten-Systems.

5.3 KI-gestützte Entwicklung

Ein weiterer, wichtiger Ansatz in der Methodik dieser Arbeit ist die aktive Einbindung von Google Gemini 3.1 Pro in den Softwareentwicklungsprozess. Entsprechend dem Paradigma des AI-Assisted Software Engineering wurde das Tool iterativ als "Copilot"genutzt, um die Entwicklerproduktivität zu steigern [11].

Der Autor nahm in diesem Projekt primär die Rollen des *Requirements Engineers* und *Solution Architects* ein, während die KI beim Prototyping, Brainstorming und bei repetitiven Coding-Aufgaben unterstützt hat. Die konkrete Aufgabenteilung definierte sich wie folgt:

- **Menschlicher Lead (Architektur, Infrastruktur und UX):** Die strategische Systemkonzeption, die Definition der 4-Schichten-Architektur sowie der Entwurf der zugrundeliegenden Infrastruktur und Ordnerstruktur oblagen der Zuständigkeit des Autors. Ein besonderer Fokus lag auf der Konzeption und weiten Teilen der manuellen Implementierung des Telegram-Frontends (`chatbot.py`). Als QA-Tester definierte der Autor zudem die Testmetriken und evaluierte die Ergebnisse kritisch.

- **KI-gestütztes Bootstrapping (LangGraph und Prompting):** Aufgrund der hohen Komplexität von LangGraph übernahm die KI das initiale Prototyping und half dem Autor dabei, sich zügig mit dem Framework zurechtzufinden. Der allererste Systemprototyp ab der Graphen-Einführung wurde maßgeblich durch Gemini mitgestaltet und anschließend vom Autor orchestriert und angepasst. Ebenso lieferte die KI starke Unterstützung beim iterativen Prompt-Engineering der frühen Testphase.

- **Operative Code-Hygiene:** Auf der operativen Ebene übernahm die KI wiederkehrende, zeitintensive Aufgaben fast vollständig. Dazu gehörten die lückenlose Kommentierung des Quellcodes, der Aufbau der Logging-Infrastruktur, das Debugging sowie die UI-Formatierung im Frontend (z. B. der Einsatz von Emojis).

Die KI fungierte nicht als autonomer Entwickler, sondern als nützliches Werkzeug, das unter menschlicher Weisung unterstützt hat. Durch die konsequente Auslagerung von syntaktischer Fleißarbeit konnte die Entwicklungszeit deutlich verkürzt werden, womit es möglich war die gewonnene Zeit direkt in die Schärfung des Gesamtsystems fließen zu lassen.

Hinweis zur KI-gestützten Textredaktion

Neben der technischen Softwareentwicklung wurde Generative KI (Google Gemini 3.1 Pro) auch im iterativen Schreibprozess dieser Arbeit als unterstützendes Werkzeug eingesetzt. Entsprechend der Leitlinien zur guten wissenschaftlichen Praxis im Umgang mit KI-Technologien fungierte das Sprachmodell hierbei ausschließlich als digitales Lektorat. Der Einsatz beschränkte sich auf die Überprüfung der Orthografie, die sprachliche Glättung sowie die stilistische Feinjustierung der manuell verfassten Textentwürfe. Die inhaltliche Strukturierung, die wissenschaftliche Argumentation, die Auswertung der Testreihen sowie sämtliche Schlussfolgerungen stellen die alleinige geistige Eigenleistung des Autors dar.

6 Evaluierung

Nach der vollständigen Implementierung der Architektur bestand der abschließende Schritt in der Evaluierung des Systems. Ziel war es also zu prüfen, inwiefern die in der Gap-Analyse identifizierten Schwächen der Vorgängerversion durch den Neuentwurf neutralisiert wurden. Alle generierten Blogposts und Protokolle der durchgeführten Testreihen sind zur vollständigen Transparenz im öffentlichen GitHub-Repository des Projekts dokumentiert [12].

6.1 Definition und Durchführung der Testreihe

Um die Qualität und Performance der neu konzipierten Multi-Agenten-Architektur evaluierbar zu machen, wurde eine Testreihe konzipiert, die möglichst viele Benchmarks auswerten kann. Anstatt das System lediglich mit generischen Prompts zu prüfen, wurden einige spezifische Testfälle definiert. Diese Testfälle unterschieden sich maßgeblich durch die RAG-Konstellation. Die Differenzierung erfolgte nach folgenden Zielsetzungen:

- **Aktualitäts-Check (Nur Websuche):** Evaluation, ob das System bei aktuelleren Themen (wie neuen Software-Releases) auf veraltetes Trainingswissen zurückgreift oder neuere Web-Informationen korrekt extrahiert.
- **Compliance-Check (Nur Dokumenten-Upload):** Ein gezielter Test zur Halluzinationsunterdrückung. Das System wurde mit einer fiktiven, logisch absurden Richtlinie konfrontiert (z. B. dem Verbot von Sauerstoff-produzierenden Pflanzen). Ziel war es zu prüfen, ob der *Fact-Checker* das vorgegebene Dokument strikt über das allgemeine Weltwissen des LLMs priorisiert.
- **Synthese-Test (Websuche und Dokument):** Die anspruchsvollste Metrik. Hier musste das System externe Optionen aus dem Web (z. B. reale Restaurant-Empfehlungen) mit den harten Restriktionen eines

lokalen Dokuments (z. B. fiktive Budget-Limits) fehlerfrei abgleichen und filtern.

- **Baseline-Messung (Ohne externe Quellen):** Ein reiner Kreativitäts-Test zur Bewertung der strukturellen und sprachlichen Qualität des Modells (Strukturierung durch den *Editor*, Formulierung durch den *Writer*) ohne die kognitive Belastung durch einen RAG-Kontext.

Die methodische Definition dieser isolierten RAG-Konstellationen war für das Benchmarking der Architektur essenziell. Nur durch die gezielte Trennung der Informationsquellen ließ sich evaluieren, ob bei Prompts, den Modellen oder der Workflow-Logik Anpassungen vorgenommen werden sollten. Basierend auf diesen vier grundlegenden Kategorien wurden für die finale Evaluierung insgesamt sieben spezifische Testfälle definiert und sowohl in der alten CrewAI-Architektur als auch im neuen LangGraph-System durchgeführt. Um die Qualität der Ergebnisse objektiv vergleichen zu können, wurden für die tabellarische Auswertung folgende Metriken festgelegt:

- **Zeit:** Die absolute Bearbeitungsdauer vom Start der Anfrage bis zur Ausgabe des finalen Dokuments. Erzwungene Systemabbrüche wurden als Fehler gewertet.
- **Revision:** Die Anzahl der vom System autonom durchgeführten Korrekturschleifen.
- **Vorgaben:** Die Instruktionstreue des LLMs hinsichtlich der Nutzerparameter, gemessen auf einer Skala von 0 bis 5.
- **Aufbau:** Die strukturelle Qualität des Endprodukts (sinnvolle Absätze, Überschriften, Formatierungen), bewertet auf einer Skala von 0 bis 5.
- **RAG:** Der Grad der erfolgreichen und korrekten Extraktion der vorgegebenen Fakten aus den Tools, bewertet auf einer Skala von 0 bis 5.
- **Halluzination:** Das Maß an inhaltlich erfundenem oder durch Weltwissen verfälschtem Kontext, kategorisiert in *Hoch, Mittel, Gering, nein.*
- **Meta-Bleeding:** Das unerwünschte Durchsickern von internen System-Anweisungen, Agenten-Gedanken oder Platzhaltern in das finale Dokument, kategorisiert in *Hoch, Mittel, Gering, nein.*

Anhand dieser standardisierten quantitativen Messwerte ließ sich die alte Architektur (CrewAI) abschließend präzise gegen den aktuellen deterministischen Graphen (LangGraph) vergleichen:

Testcase	Zeit (Min)	Revision	Vorgaben	Aufbau	RAG	Halluzination	Meta-Bleeding
1	54	N/A	2/5	1/5	0/5	Hoch	Hoch
2	19	N/A	3/5	2/5	1/5	Hoch	Hoch
3	62	N/A	1/5	3/5	0/5	Hoch	nein
4	57[1]	N/A	0/5	0/5	-	Hoch	Hoch
5	57[1]	N/A	0/5	0/5	0/5	Hoch	nein
6	59[1]	N/A	0/5	0/5	0/5	Hoch	nein
7	12	N/A	0/5	0/5	0/5	Hoch	Mittel

[1] Systemabbruch (Agent Death-Spiral) ohne finale Output-Generierung.

Tabelle 1: Quantitative Evaluationsergebnisse unter CrewAI

Testcase	Zeit (Min)	Revision	Vorgaben	Aufbau	RAG	Halluzination	Meta-Bleeding
1	11	0	3/5	5/5	0/5	Hoch	nein
2	11	1	2.5/5	5/5	1/5	Mittel	nein
3	7	1	4.5/5	5/5	5/5	nein	nein
4	9	0[2]	3/5	5/5	-	Mittel	nein
5	14	0[2]	4/5	5/5	4/5	Gering	nein
6	9	0[2]	4/5	5/5	5/5	Hoch	nein
7	10	0	5/5	5/5	5/5	nein	nein

[2] Revisionsschleife wurde aufgrund eines Routing-Fehlers im Graphen nicht ausgelöst.

Tabelle 2: Quantitative Evaluationsergebnisse unter LangGraph

6.2 Backend

Die Auswertung der Testfälle belegt, dass der Wechsel der Architektur von CrewAI zu LangGraph die fundamentalen Backend-Defizite der Vorgängerversion erfolgreich behebt. Während komplexe Aufgaben unter CrewAI regelmäßig in unkontrollierbaren Endlosschleifen mündeten und Systemabbrüche nach teils einstündiger Laufzeit erzwangen, verläuft der Workflow nun deterministisch. Die Bearbeitungszeiten wurden drastisch auf durchschnittlich zehn Minuten reduziert, und das unerwünschte "Meta-Bleeding" konnte durch das strikte State-Management vollständig eliminiert werden. Zudem zeigte sich das System bei logischen Restriktionen (wie dem Budget-Limit in Testfall 3) äußerst verlässlich.

Dennoch offenbart die kritische Betrachtung der Ergebnisse, dass die neue Architektur eigene Schwachstellen mit sich bringt. Zwar detektierte der neu eingeführte Fact-Checker inhaltliche Halluzinationen präzise, jedoch offenbarte sich in den Testfällen 4, 5 und 6 ein kritischer Fehler im Routing. Weil die Python-Logik den unstrukturierten Text-Output des LLMs nicht immer fehlerfrei interpretieren konnte, wurden zwingend notwendige Revisionsschleifen trotz erkannter Fehler ignoriert und die Texte ungeprüft an den Polisher weitergereicht.

Darüber hinaus zeigte insbesondere Testfall 2, dass auch ein expliziter Graph das grundlegende Problem der LLM-Halluzination nicht vollständig heilt: Konfrontiert mit logisch absurden RAG-Dokumenten neigte das Modell weiterhin dazu, die bereitgestellte „Wahrheit“ heimlich durch plausibler klingendes Weltwissen zu glätten. Auch bei unzureichendem Quellkontext (Testfall 6) kompensierte das Modell fehlende Fakten durch das Erfinden von Zitaten. Zusam-

menfassend lässt sich festhalten, dass LangGraph die Unberechenbarkeit autonomer Agenten zwar erfolgreich durch technische Transparenz und Kontrolle ersetzt, die Fehleranfälligkeit des Systems dadurch jedoch robusterer Routing-Konditionen bedarf.

Ein weiterer Erfolgsfaktor ist der Umbau vom ineffizienten "One-Size-Fits-All"-Ansatz der Vorgängerversion, bei der ein einziges Modell (Llama 3.1) alle Aufgaben übernahm. Die neue Architektur nutzt stattdessen eine rollenspezifische *LLM-Factory*. Analytische Knoten (wie Researcher und Fact-Checker) greifen nun auf ein auf Logik optimiertes Modell (Qwen) zurück, Textgenerierung erfolgt hingegen durch ein kreatives Modell (Gemma). Außerdem ist es nun möglich, mit dem Parameter *temperature* die Flexibilität der Modelle auf den jeweiligen Agenten zu steuern.

6.3 Frontend

Neben den Problemen im Backend litt der Prototyp unter massiven Einschränkungen der Usability. Die Anwendung war für den User intransparent und schwer bedienbar, da die Nachrichten des Bots ohne Highlights und Absätze die Lesbarkeit erschwert haben und Konfigurationen mühsam in Einzelschritten als Freitext in den Chat abgetippt werden mussten. Zudem führte die im Durchschnitt hohe Verarbeitungsdauer gepaart mit mangelnder Transparenz über den Status des Prozesses (Nachricht durchgehend nur 'processing...') dazu, dass nicht absehbar war, ob und wann der Post fertig werden würde. Die Eingabeaufforderung wurde daher durch einen intuitiven Wizard-Prozess mit klickbaren Telegram-Inline-Keyboards ersetzt, was sowohl die Lesbarkeit als auch die Bedienbarkeit deutlich steigern konnte. Das Problem der fehlenden Transparenz des Workflows wurde ebenfalls behoben, denn der Nutzer erhält nun kontinuierliche Status-Updates direkt im Chat. Der gesamte Prozess wird somit begleitet durch eine nachvollziehbare, visuelle Darstellung, was die User Experience (UX) auf ein professionelles Niveau hebt.

7 Adaptation Manual

Hinweis: Da sich der Quellcode und die Architektur dieses Projekts an internationale Open-Source-Standards richten, wurde die technische Systemdokumentation im Original auf Englisch verfasst und im Folgenden auf Deutsch übersetzt. Das folgende Kapitel entspricht inhaltlich dem offiziellen Adaptation Manual aus dem zugehörigen GitHub-Repository [12] und dient als technischer Leitfaden zur Modifikation des Systems.

Eines der primären Entwicklungsziele dieses Projekts ist die Anpassungsfähigkeit. Die Systemarchitektur trennt bewusst **Konfiguration**, **Workflow-Logik**, **Agentenverhalten**, **Modellauswahl**, **Retrieval (RAG)** und **Schnittstelle (Interface)** voneinander. Dieses Handbuch erklärt nicht nur, *was* angepasst werden kann, sondern auch, *wie* typische Modifikationen auf stabile Weise implementiert werden können.

7.1 Schnelle Orientierung

Wichtige Anpassungsbereiche (am stabilsten → am invasivsten):

1. **Prompts & Vorlagen** (`config/agents.yaml`, `config/tasks.yaml`)
2. **Globale Einstellungen** (`config/configs.yaml`, `.env`)
3. **Workflow-Topologie & Routing** (`graph/workflow.py`)
4. **Knoten-Logik / Zustands-Schema** (`graph/nodes.py`)
5. **Modell-Fabrik & Provider-Wechsel** (`llm/factory.py`)
6. **Retrieval-Pipeline** (`tools/vectorstore.py`, `tools/search_tool.py`)
7. **Telegram UX / Konversationszustände** (`chatbot.py`, `states.py`)

7.2 Anpassung der Konfiguration (config-Ordner)

Der `config/`-Ordner ist die deklarative Steuerungsebene. Er ermöglicht Änderungen, ohne ausführbaren Code anzufassen, und ist der bevorzugte Startpunkt. `factory.py` ruft die Modelle automatisch ab.

7.2.1 `config/configs.yaml` - Globale Laufzeiteinstellungen

Zweck:

- Standardmodelle (LLM + Embeddings)

Wie man anpasst: Modellwechsel:

```
llm_model: llama3.1:8b-instruct-q4_K_M
embedding_model: mxbai-embed-large
```

7.2.2 `config/agents.yaml` - Agentenrollen, Persona, Einschränkungen

Zweck:

- Definiert die Identität und die Verhaltensgrenzen jedes Agenten durch einen System-Prompt oder Rollen-Prompt.

Typische Agenten:

- Researcher
- Editor
- Writer
- FactChecker
- Polisher

Wie man anpasst: Beispiel: Einführung von Anweisungen zum Zitierstil (Polisher):

```
polisher:
  system_prompt: |
    If sources are provided in the briefing,
    add inline citations like [1], [2].
    Do not invent sources.
```

7.2.3 `config/tasks.yaml` - Schritt-Outputs & Lieferobjekte

Zweck:

- Legt fest, was jeder Schritt produzieren muss (Struktur, Checklisten, Formatierungsvorgaben).

Wie man anpasst: Den Editor dazu bringen, ein striktes Gliederungsformat auszugeben:

```
editor_task:
  description: |
    Create an outline with numbered H2 sections and
    bullet points per section.
    Include supporting evidence placeholders.
```

7.3 Anpassung des Workflows (LangGraph)

Der Workflow ist definiert in:

- `graph/workflow.py`

Knoten (Nodes) sind Verarbeitungsschritte, Kanten (Edges) definieren die Ausführungsreihenfolge. LangGraph ermöglicht bedingtes Routing (Revisionsschleifen).

7.3.1 Ändern des Agenten-Ablaufs

Wie man anpasst: Einen neuen Knoten hinzufügen (Beispiel: SEO-Schritt nach dem Polisher):

```
workflow.add_node("seo", seo_node)
workflow.add_edge("polisher", "seo")
workflow.add_edge("seo", END)
```

Einen Knoten entfernen:

- `add_node(...)` löschen
- Die Kanten neu verbinden, damit der Graph zusammenhängend bleibt

7.3.2 Ändern der Revisionslogik

Die Revisionsschleife wird durch bedingte Kanten ausgehend vom FactChecker-Knoten gesteuert.

Mögliche Änderungen:

- Erhöhen der maximalen Revisionsanzahl
- Anpassen der PASS/FAIL-Kriterien

Wie man anpasst: Revisionsanzahl erhöhen:

- Identifizieren, wo `revision_count` inkrementiert und verglichen wird.

7.4 Anpassung des Agentenverhaltens

Die Agentenlogik ist implementiert in:

- `graph/nodes.py`

Jeder Knoten kapselt eine Rolle:

- `research_node`
- `editor_node`
- `writer_node`
- `fact_check_node`
- `polisher_node`

7.4.1 Der gemeinsame Zustand (Informationsfluss)

LangGraph-Knoten tun typischerweise Folgendes:

- aus dem `state` (Zustand) lesen (Inputs, die von früheren Knoten produziert wurden)
- neue Schlüssel (Keys) zurück in den `state` schreiben

Wie man anpasst: Ein neues Zustandsfeld hinzufügen (Beispiel: `citations`):

1. Im `research_node` Zitate sammeln und speichern:
   - `state["citations"] = ...`
2. Nachfolgende Prompts aktualisieren, um Zitate einzubeziehen.
3. Im `polisher_node` die Zitierformatierung erzwingen.

7.4.2 Verbesserungen des Writers (wie Neuschreibungen implementiert werden)

Typisches Muster für Neuschreibungen (Rewrites):

- `critique` und `previous_draft` in den Prompt aufnehmen
- den Writer anweisen, nur notwendige Änderungen vorzunehmen

Wie man anpasst: Eine 'Diff-Style'-Neuschreibungs-Disziplin hinzufügen:

- den Writer anweisen, Abschnitte unverändert zu lassen, es sei denn, sie sind markiert
- oder ein kurzes Änderungsprotokoll (Change Log) erstellen lassen

7.5 Modellanpassung (`llm/factory.py`)

Die Modellkonfiguration wird gehandhabt über:

- `llm/factory.py`

  Zweck:

- Rollenspezifische Spezialisierung (logisch vs. kreativ)

7.5.1 Wechseln der LLM-Modelle (Ollama)

Neues Modell herunterladen:

```
ollama pull <model_name>
```

Danach in `config/configs.yaml` ändern.

Wie man anpasst:

- `temperature` ändern, um die Flexibilität der Agenten anzupassen
- `keep_alive` ändern, um zu steuern, wie lange das Modell im Speicher geladen bleibt

7.5.2 Verwendung von Nicht-Ollama-Modellen

Um den Provider zu wechseln:

1. Die `Ollama`-Initialisierung durch den gewünschten Provider-Wrapper ersetzen
2. Sicherstellen, dass die Ausgabeformate mit den nachfolgenden Knoten kompatibel bleiben

7.6 RAG-Anpassung

Retrieval-Komponenten sind definiert in:

- `tools/vectorstore.py`
- `tools/search_tool.py`

7.6.1 Anpassung des Vektorspeichers (Chroma)

Typische Parameter zur Feinabstimmung:

- `chunk_size`
- `chunk_overlap`
- Top-k-Retrieval

Wie man anpasst:

- Kleinere Chunks verbessern die Präzision bei faktischen Anfragen
- Größere Chunks verbessern die Kohärenz beim Schreiben längerer Texte

Ein gängiger Weg zur Feinabstimmung:

1. Die Überlappung (overlap) erhöhen, wenn wichtige Fakten aufgeteilt werden
2. `k` erhöhen, wenn den Antworten wichtiger Kontext fehlt

7.6.2 Anpassung der Websuche

Das Suchwerkzeug macht typischerweise Folgendes:

- frägt einen Suchanbieter ab (DuckDuckGo)
- filtert die Ergebnisse nach Domain / Dateityp

Wie man anpasst:

- Blacklist/Whitelist erweitern
- autoritative Domains bevorzugen (z. B. `.edu`, offizielle Dokumente)
- einen Aktualitäts-Bias (recency bias) hinzufügen, wenn das Thema zeitkritisch ist

7.7 Anpassung des Telegram-Chatbots

Die Konversationslogik ist definiert in:

- `chatbot.py`

Um den Interaktionsfluss anzupassen:

1. Konversationszustände modifizieren
2. Frage-Sequenz anpassen
3. Handler und Übergänge aktualisieren

Wie man anpasst:

- Neue Konfigurationsparameter hinzufügen durch:
  - Hinzufügen eines neuen Zustands (`states.py`)
  - Aktualisieren der Zustandsautomaten-Übergänge
  - Übergeben des neuen Feldes an die Workflow-Inputs
- UI-Stil ändern:
  - Freitexteingabe durch Inline-Buttons ersetzen

8 Fazit und Ausblick

Die vorliegende Bachelorarbeit befasste sich mit der Konzeption, Neuentwicklung und Evaluierung eines agentenbasierten Chatbots zur automatisierten Erstellung von Blogposts. Ausgangspunkt der Arbeit war die genaue Gap-Analyse eines bestehenden Prototyps, der auf dem Framework CrewAI basierte. Es zeigte sich vor allem durch Intransparenz, Ineffizienz und eine hohe Anfälligkeit für Halluzinationen, dass derart abstrahierte Frameworks für deterministische und fehlerresistente Aufgabenfolgen weniger geeignet sind, als ein System, welches auf einer Logik durch Graphen definiert ist.

8.1 Zusammenfassung der Ergebnisse

Der Wechsel zu der graphenbasierten Architektur von LangGraph und das neu gestaltete Telegram-Interface markieren den zentralen Erfolg dieses Projekts. Durch das explizite State-Management und die Einführung eines zyklischen *Fact-Checker-Loops* konnte das Systemverhalten signifikant stabilisiert werden. Die Auswertung der definierten Testreihen belegt, dass die neue Architektur schneller, genauer und transparenter arbeitet.

Die UX wurde durch das neu konzipierte Telegram-Frontend auf ein professionelleres Niveau gehoben. Der Ersatz freier Texteingaben mit klickbaren

Inline-Keyboards und die Einführung von visuellen Highlights und des Status-Streamings eliminierten die kognitive Überlastung während der Textgenerierung vollständig. Das resultierende System ist somit nicht nur im Backend robuster, sondern auch im Frontend intuitiv und nutzerzentriert bedienbar.

8.2 Ausblick

Obgleich das entwickelte Multi-Agenten-System die gesetzten Ziele weitgehend erfüllt, bietet die gewählte modulare Architektur eine ideale Grundlage für zukünftige Erweiterungen.

Ein Möglicher Ausgangspunkt ist die technische Härtung der Routing-Logik in der *Fact-Checker*-Schleife. Da die aktuelle Auswertung der *Conditional Edges* auf fehleranfälligem String-Matching von unstrukturiertem Text basiert, sollten zukünftige Iterationen zwingend auf *Structured Outputs* umgestellt werden.

Ein weiterer möglicher Ansatzpunkt für die Weiterentwicklung ist die Integration von *Human-in-the-Loop*-Mechanismen (HITL). Da LangGraph den Systemzustand nach jedem Knotenpunkt pausieren und speichern kann, ließe sich das Telegram-Frontend dahingehend erweitern, dass der Nutzer selbst eine Rolle während des Workflows spielt und beispielsweise das generierte Outline des *Editor-Agenten* manuell absegnet oder per Chat-Nachricht korrigiert, bevor der *Writer-Agent* den finalen Text ausformuliert.

Zudem birgt die *LLM-Factory* das Potenzial für umfassende Modell-Benchmarks. Da die Logik komplett entkoppelt ist, könnten in zukünftigen Arbeiten gezielt kleinere, quantisierte Open-Source-Modelle gegen Giganten (wie GPT-4o) im direkten Zusammenspiel der Agenten-Rollen evaluiert werden.

Zusammenfassend demonstriert diese Arbeit, dass die erfolgreiche Implementierung von Generativer KI nicht allein von der Wahl des größten Sprachmodells abhängt, sondern maßgeblich von einer transparenten, durchdachten Softwarearchitektur und einer konsequenten Ausrichtung am Endanwender.

Literatur

[1] Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Beshi, B., Suh, P., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., et al. Guidelines for human-ai interaction. In *Proceedings of the 2019 CHI conference on human factors in computing systems* (2019), pp. 1–13.

[2] CrewAI. Crewai: Framework for orchestrating role-playing, autonomous ai agents. `https://docs.crewai.com/`, 2024. Zugegriffen: 24. Februar 2026.

[3] CrewAI Community. [bug] authentication error when using openai compatible llms - issue #2647. `https://github.com/crewAIInc/crewAI/issues/2647`, 2025. GitHub Repository. Zugegriffen: 25. März 2026.

[4] Dwivedi, Y. K., Kshetri, N., Hughes, L., Slade, E. L., Jeyaraj, A., Kar, A. K., Baabdullah, A. M., Koohang, A., Raghavan, V., Ahuja, M., et al. "so what if chatgpt wrote it?" multidisciplinary perspectives on opportunities, challenges and implications of generative conversational ai for research, practice and policy. *International Journal of Information Management* (2023).

[5] Følstad, A., and Brandtzæg, P. B. Chatbots and the new world of hci. *interactions 24*, 4 (2017), 38–42.

[6] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., and Wang, H. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* (2023).

[7] IBM Developer. Comparing ai agent frameworks: Crewai, langgraph, and beeai. `https://developer.ibm.com/articles/awb-comparing-ai-agent-frameworks-crewai-langgraph-and-beeai/`, 2025. Zugegriffen: 25. März 2026.

[8] LangChain. Langgraph: Build resilient language agents as graphs. `https://langchain-ai.github.io/langgraph/`, 2024. Zugegriffen: 22. Februar 2026.

[9] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems 33* (2020), 9459–9474.

[10] Luger, E., and Sellen, A. "like having a really bad pa: The gulf between user expectation and experience of conversational agents. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems* (2016), ACM, pp. 5286–5297.

[11] Peng, S., Kalliamvakou, E., Cihon, P., and Demirer, M. The impact of ai on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590* (2023).

[12] Rodenbäck, M. Baragmaschatbot-2.0-langgraph. `https://github.com/Marrody/BaRagmasChatbot-2.0-LangGraph`, 2026. Zugegriffen: 26. Februar 2026.

[13] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems 36* (2024).

[14] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems 35* (2022), 24824–24837.

[15] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, H., Liu, J., et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155* (2023).

[16] Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., et al. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023).